

RI.UVT.RO Redesign

Timeline Synthesis

Team Subgroup Architecture
Headless WordPress + React Implementation

Focus: Mapping the implementation phases, team roles, subgroup responsibilities, and concrete outputs into an actionable 8-week production timeline.

Technology Stack: WordPress CMS + REST API + React + Vite +
TypeScript
Team Q-UW9

Contents

1	Document Purpose	2
2	Execution Logic	2
3	Team Subgroups	2
3.1	Team A — Coordination, UX, and Structure	3
3.2	Team B — Visual System and Frontend Foundation	3
3.3	Team C — WordPress Content Systems and API Structure	3
3.4	Team D — Quality, Accessibility, and Final Stabilization	4
4	Suggested Timeline	4
5	Phase-to-Team Mapping	5
6	Detailed Weekly Role Distribution	5
6.1	Weeks 1–2: Definition Sprint	5
6.2	Weeks 3–4: System-Building Sprint	6
6.3	Weeks 5–6: Content and Dynamic Build Sprint	6
6.4	Weeks 7–8: Validation and Finalization Sprint	6
7	Operational Collaboration Rules	7
8	Final Delivery Checklist	7
9	Conclusion	8

1. Document Purpose

This synthesis turns the project roles and implementation phases into a coordinated production schedule.

It answers three practical questions:

- **When** each stream of work begins and intensifies;
- **Which roles work together** as subteams;
- **What concrete outputs** should exist by the end of each stage.

The structure assumes an **8-person team** working in parallel on a **Headless WordPress** + **React** implementation.

2. Execution Logic

The implementation follows the same build philosophy defined in the proposal and onboarding documents:

- the website is rebuilt as a **real digital product**, not only visually restyled;
- WordPress remains the content-management layer;
- React becomes the public-facing frontend;
- the REST API connects the two systems;
- work progresses from audit and structure to system building, content population, validation, and optimization;
- every role contributes directly to implementation.

Core rule: collaboration happens through subgroups of teams, not isolated individual work.

3. Team Subgroups

3.1 Team A — Coordination, UX, and Structure

Roles:

- **Role 1:** Project Manager + Frontend Lead
- **Role 2:** UX Researcher + Frontend Contributor
- **Role 3:** Information Architect + WordPress Structure Specialist

Mission: Define the strategic direction of the build: user flows, sitemap, content hierarchy, route logic, technical rules, page relationships, and implementation priorities.

Main outputs: audit findings, user flows, sitemap, page hierarchy, navigation model, structure decisions, and task prioritization.

3.2 Team B — Visual System and Frontend Foundation

Roles:

- **Role 4:** UI Designer + Frontend Implementer
- **Role 5:** React Developer — Pages and Components
- **Role 1:** Project Manager + Frontend Lead

Mission: Turn the agreed structure into a coherent frontend system: layout shell, typography, spacing, reusable UI components, responsive page templates, and visual consistency.

Main outputs: design system, homepage and internal page layouts, shared React components, route structure, responsive page shells.

3.3 Team C — WordPress Content Systems and API Structure

Roles:

- **Role 6:** WordPress Developer — API and Content Types
- **Role 7:** Content Strategist
- **Role 3:** Information Architect + WordPress Structure Specialist

Mission: Convert the static structure into a scalable editorial system: pages, posts, custom post types, custom fields, structured content, and REST API-ready data.

Main outputs: custom post types, editable content models, populated pages, cleaned text, reusable content patterns, API-ready backend structure.

3.4 Team D — Quality, Accessibility, and Final Stabilization

Roles:

- **Role 8:** QA, Accessibility, Performance, and Frontend Fixes
- **All other roles:** issue resolution in their own area of ownership

Mission: Test, validate, optimize, and stabilize the platform continuously.

Main outputs: bug lists, accessibility checks, responsive validation, performance optimizations, final review reports, and verified fixes.

4. Suggested Timeline

A realistic academic implementation cadence is an **8-week production cycle**. The weeks below do not imply that one phase fully ends before the next begins. They show the **dominant focus** of each period.

Week	Primary Focus	Lead Team(s)	Expected Concrete Output
1	Audit, user analysis, content inventory, project setup	Team A + Role 1	Current-site audit, problem list, user groups, initial backlog, Git repository workflow, Laragon and React development setup.
2	Sitemap, user flows, navigation logic, structural decisions	Team A	Final information architecture draft, navigation model, page grouping, decision on static pages, posts, and custom post types.
3	Design system and frontend strategy	Team B + Team A	Typography, spacing, color system, layout rules, core wireframes, route strategy, component plan.
4	WordPress foundation and React shell	Team B + Team C	WordPress content base, REST API checks, header, footer, layout wrappers, shared React components, first page templates.
5	Dynamic structures and reusable content models	Team C + Team B	Custom post types, custom fields, API-connected data structures, reusable frontend sections, content model validation.
6	Main page building and content population	Team C + Team B	Implemented homepage and key internal pages, restructured content, CTAs, student journey pages, support and contact sections.

7	UX refinement, dynamic features, testing rounds	Team D + Role 2 + Role 6	Search/filter helpers where justified, responsive fixes, accessibility review, navigation refinement, bug tracking.
8	Final QA, optimization, documentation, packaging	Team D + Role 1	Stable validated site, resolved major issues, optimized assets, final documentation, submission-ready project.

5. Phase-to-Team Mapping

Project Phase	Main Work	Lead Roles
Phase 1 — Audit and Planning	Review the existing site, identify problems, define user groups, prepare the initial execution backlog.	1, 2, 3
Phase 2 — IA and Design System	Lock sitemap, menu logic, template families, route structure, and visual system.	2, 3, 4, 1
Phase 3 — Development Foundation	Configure WordPress, React + Vite, environment variables, API access, and shared frontend shell.	1, 4, 5, 6
Phase 4 — Content Architecture	Create pages, posts, custom post types, custom fields, and reusable content structures.	3, 6, 7
Phase 5 — Frontend Implementation	Build React pages, components, dynamic views, filters, accordions, and user-facing flows.	1, 2, 4, 5, 6
Phase 6 — QA, Accessibility, Performance	Test, fix, optimize, validate, and prepare the final project package.	8, 1, 4, 5, 6, 7

6. Detailed Weekly Role Distribution

6.1 Weeks 1–2: Definition Sprint

Active priority roles: 1, 2, 3, with early contribution from 4 and 7.

What happens:

- Role 1 initializes workflow, frontend standards, branch strategy, and task ownership.
- Role 2 performs UX audit and maps user journeys.
- Role 3 defines content grouping, sitemap, hierarchy, and future WordPress structure.
- Role 4 begins visual benchmarking and prepares the visual logic for the design system.
- Role 7 starts content audit to identify overloaded, outdated, or unclear sections.

Milestone: by the end of Week 2, the team should know what the site contains, how it should be grouped, and how users should move through it.

6.2 Weeks 3–4: System-Building Sprint

Active priority roles: 1, 4, 5, 6, with structural support from 3.

What happens:

- Role 4 formalizes the design system and implements the visual language.
- Role 5 creates the React route structure, layout shell, and reusable frontend components.
- Role 6 prepares WordPress content structures and REST API exposure.
- Role 1 enforces reuse standards and keeps frontend decisions consistent.
- Role 3 verifies that implementation matches the agreed information architecture.

Milestone: by the end of Week 4, the project should have a working WordPress base, REST API connection, and React frontend shell.

6.3 Weeks 5–6: Content and Dynamic Build Sprint

Active priority roles: 6, 7, 5, with continued coordination from 1 and 3.

What happens:

- Role 6 builds custom post types, fields, reusable backend structures, and API-ready content.
- Role 7 restructures and inserts content into real pages.
- Role 5 ensures frontend templates remain flexible under real content conditions.
- Role 3 keeps content models aligned with the structural architecture.
- Role 2 reviews whether implemented journeys remain clear to users.

Milestone: by the end of Week 6, the website should contain real page content, reusable frontend sections, and a maintainable editing structure.

6.4 Weeks 7–8: Validation and Finalization Sprint

Active priority roles: 8, supported by the full team.

What happens:

- Role 8 performs responsive testing, accessibility validation, bug tracking, and performance review.
- Role 1 verifies global consistency and manages remaining priorities.

- Roles 4, 5, and 6 fix visual, frontend, and backend-content issues.
- Role 7 adjusts content clarity, CTA wording, and readability where testing reveals friction.
- Role 2 validates navigation clarity and final usability.

Milestone: by the end of Week 8, the result should be a submission-ready, tested, coherent, and maintainable website build.

7. Operational Collaboration Rules

- **Team A decides early.** Structure, hierarchy, and page logic must be stabilized before mass page building begins.
- **Team B standardizes before scaling.** The component and layout system must be defined before many pages are populated.
- **Team C populates through systems, not improvisation.** Content should be inserted using reusable layouts and structured CMS fields.
- **Team D tests continuously.** QA and accessibility should begin as soon as the first working pages exist.
- **Role 1 remains cross-functional.** This role anchors consistency between all subgroups and prevents fragmentation.

8. Final Delivery Checklist

By the end of the timeline, the team should be able to deliver:

1. A documented audit of the old website;
2. A finalized sitemap and navigation structure;
3. A defined design system;
4. A working WordPress CMS structure;
5. REST API-connected React pages;
6. Reusable frontend components;
7. Completed priority pages for the main user journeys;
8. Structured and improved content;
9. Accessibility and QA validation results;
10. Final technical and organizational documentation.

9. Conclusion

The project should be managed as a coordinated product build with overlapping specialist teams.

The timeline keeps the workflow realistic:

Understand → Structure → Build → Populate → Validate

This preserves the original role logic while turning it into an actionable execution calendar.